DRUPALCON
ASIA
2016

DRUPALCON ASIA 2016

Design Patterns in PHP

Mohit Aghera

Ankit Agrawal

axelerant

About Us

Mohit Aghera

Drupal Developer at Axelerant

Drupal : mohit_aghera

Twitter : mohit_rocks

About Us

Ankit Agrawal

Drupal Developer at Axelerant

Drupal : Ankit Agrawal

Twitter : erankyt

DRUPALCON ASIA 2016

Design patterns in PHP

- Proven object oriented experience
- They aren't invented but discovered
- Patterns doesn't gives you code !

axelerant

Why to use Design Patterns

- Shared vocabulary is powerful
- Say more with less
- Easier to discuss at high level with patterns, less discussion

The OO layer cake

OO Design patterns
Observer, iterator, decorator, strategy etc..

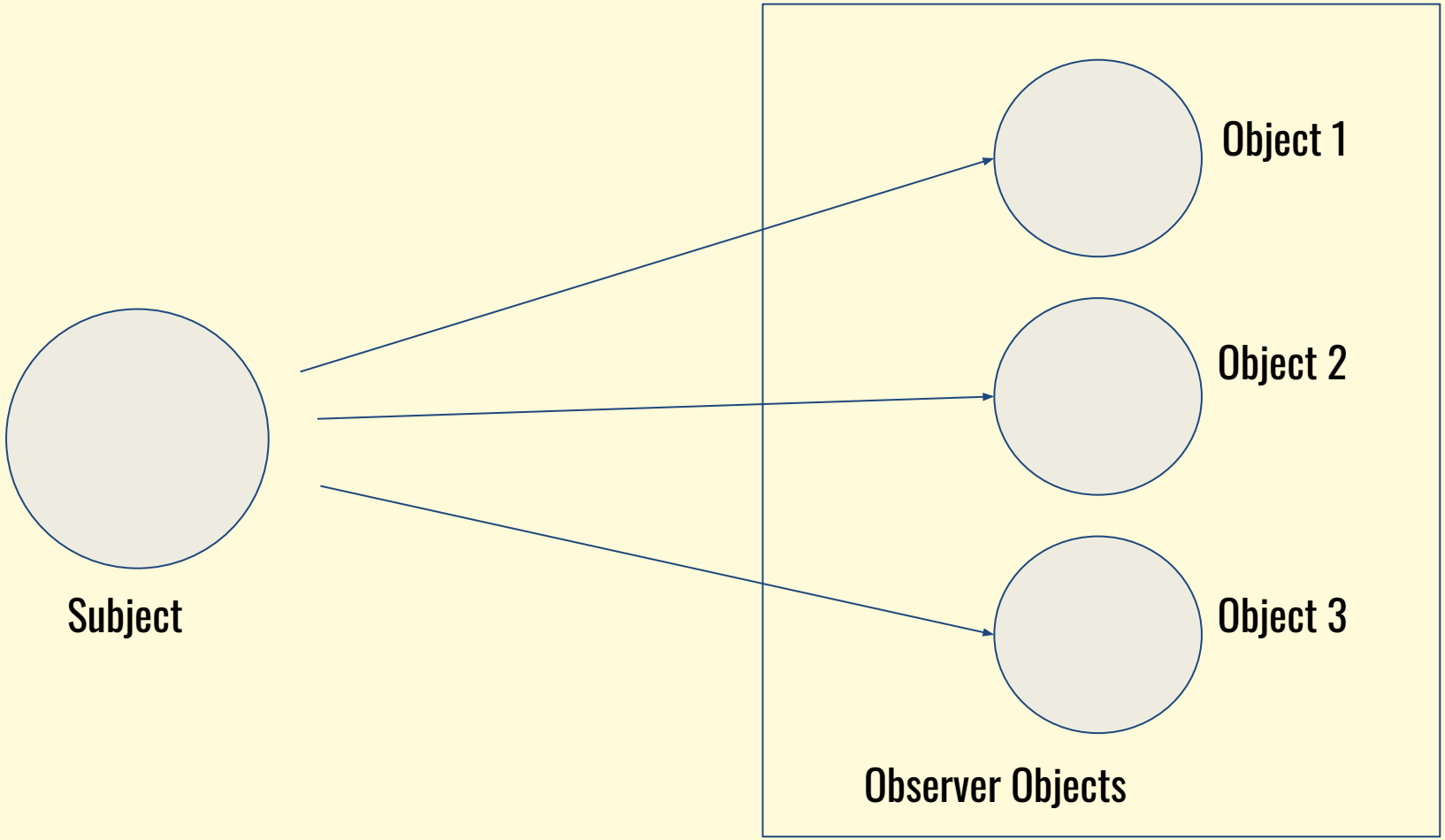
OO Principles
Encapsulate that varies, Program to interface - not implementation

OO Basics
Abstraction, Inheritance, Encapsulation, Polymorphism

Patterns types

Creational	Structural	Behavioural
Abstract Factory Singleton	Decorator Facade	Observer Visitor Strategy

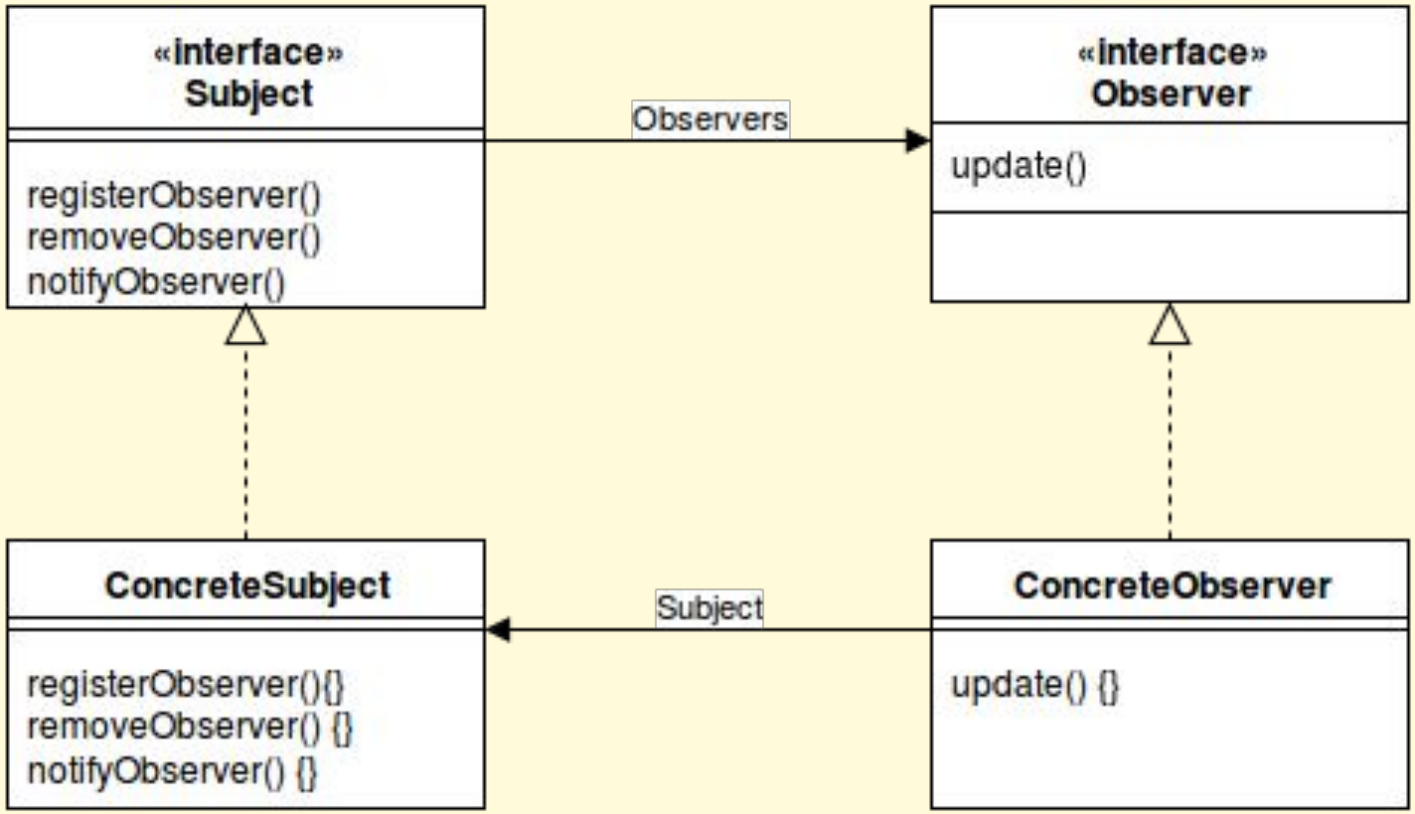
Observer Pattern



Observer Pattern

- Behavioural pattern
- Defines one to many relationship
- Dependents are notified on state change

Observer Pattern



Observer Pattern

- Loose coupling
- SplSubject and SplObserver does the job
- hook_node_load() hook_user_login()

Observer Pattern in D8

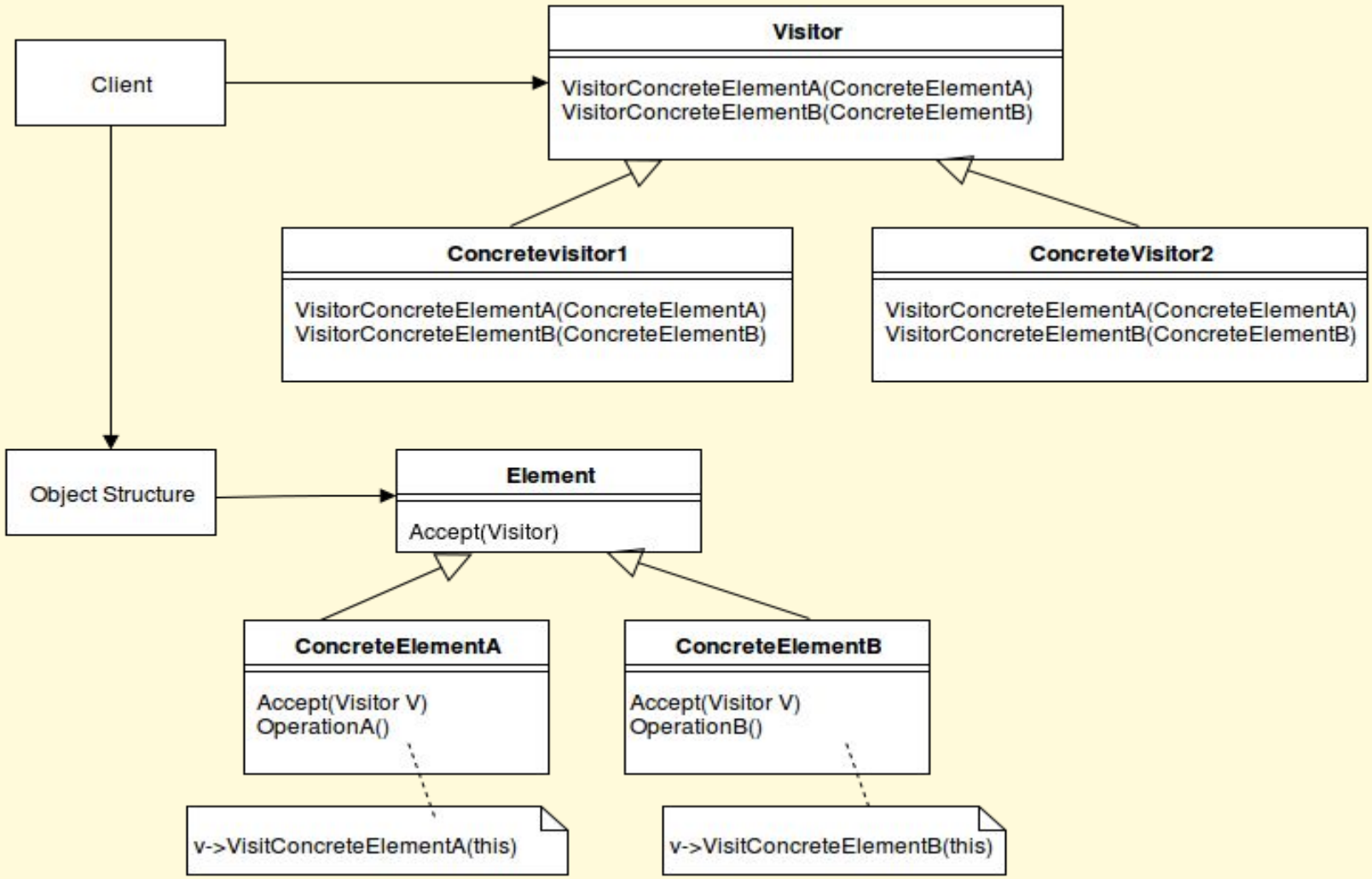
- Event dispatcher component of Symfony

Observer Pattern in D8

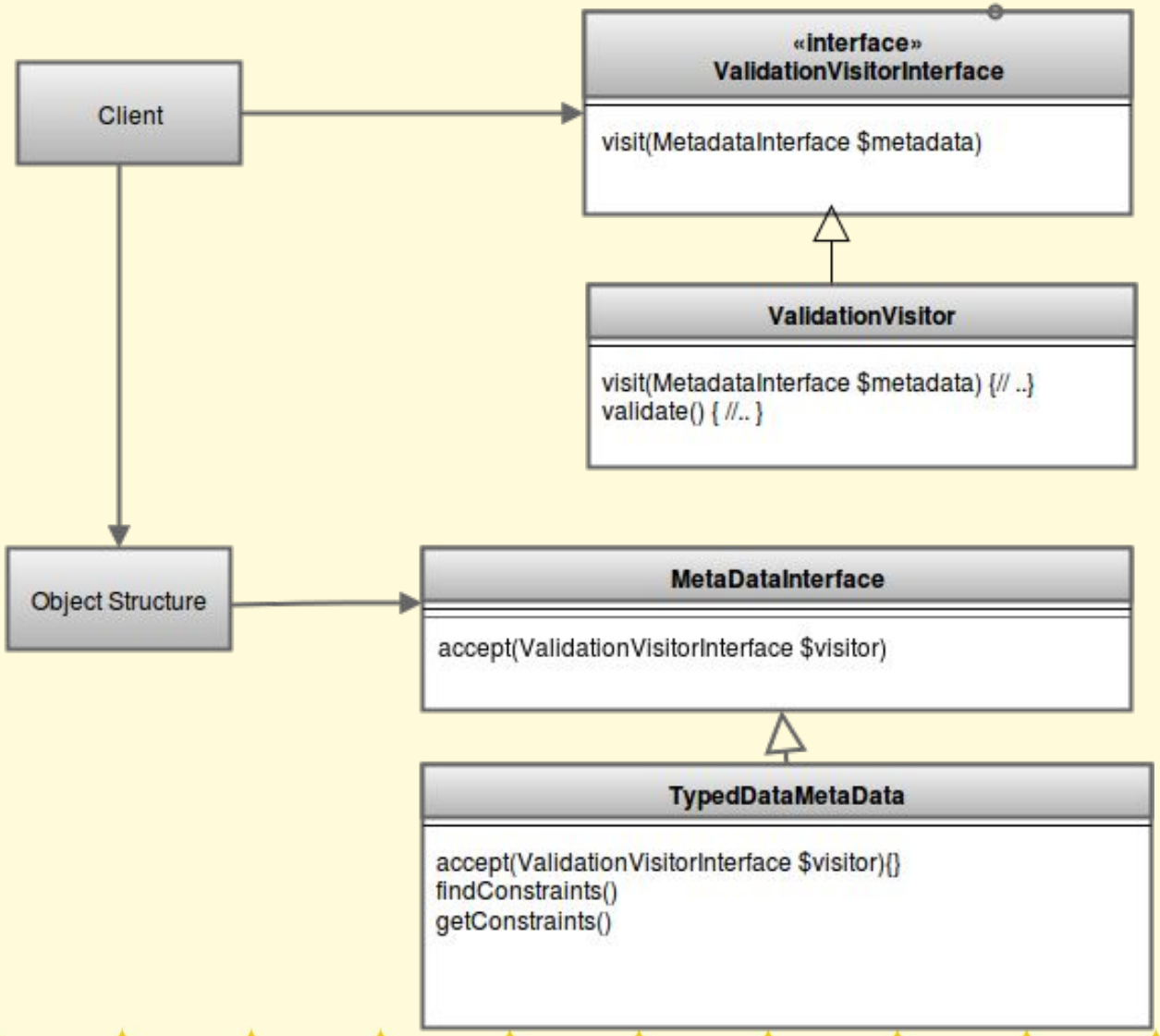
Visitor Pattern

- Behavioural pattern
- Applies to objects
- Extends behaviour of subject
- Operation to be performed on element of objects

Visitor Pattern



Visitor Pattern D8



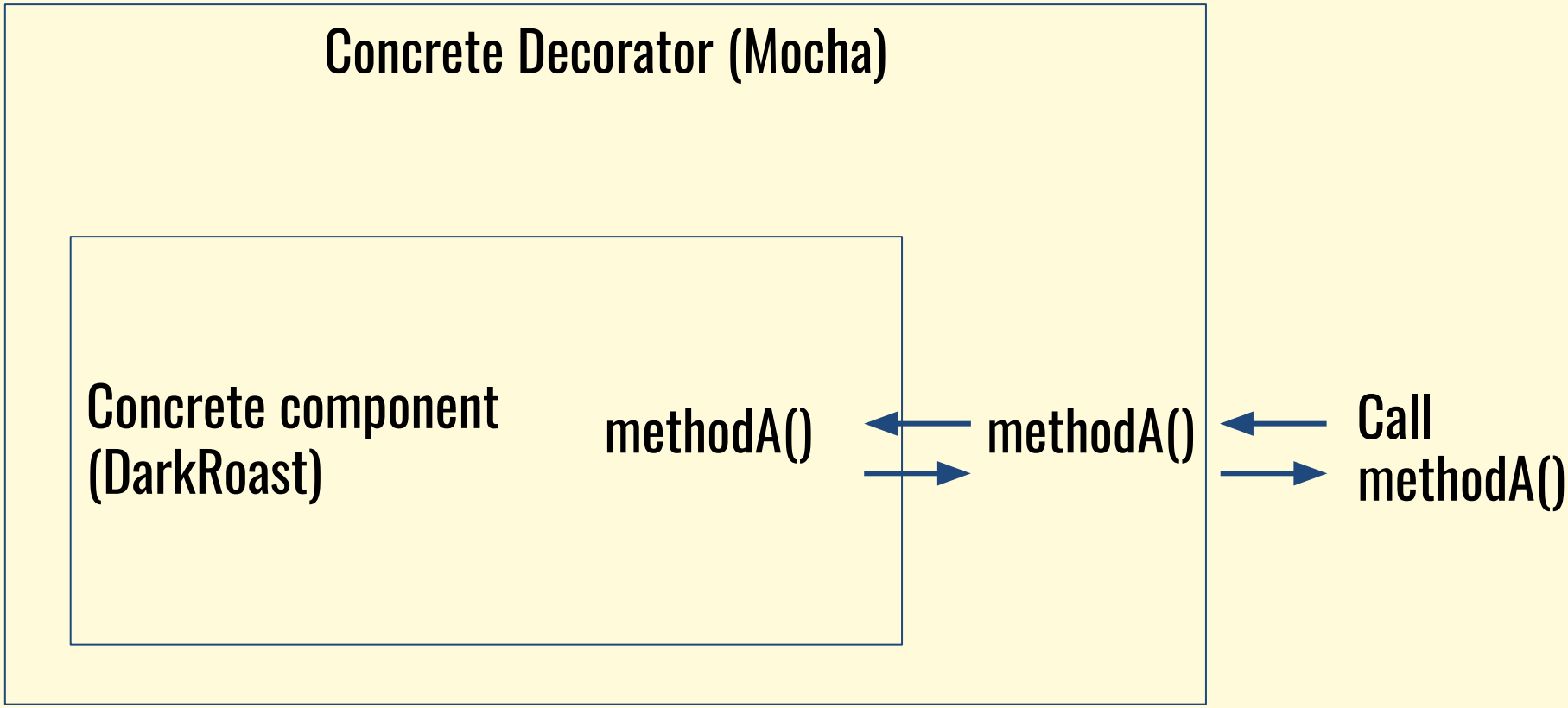
Visitor Pattern

- Makes adding new operations easy.
- Gathers related operations and separates unrelated ones
- hook_form_alter()
- hook_node_view()

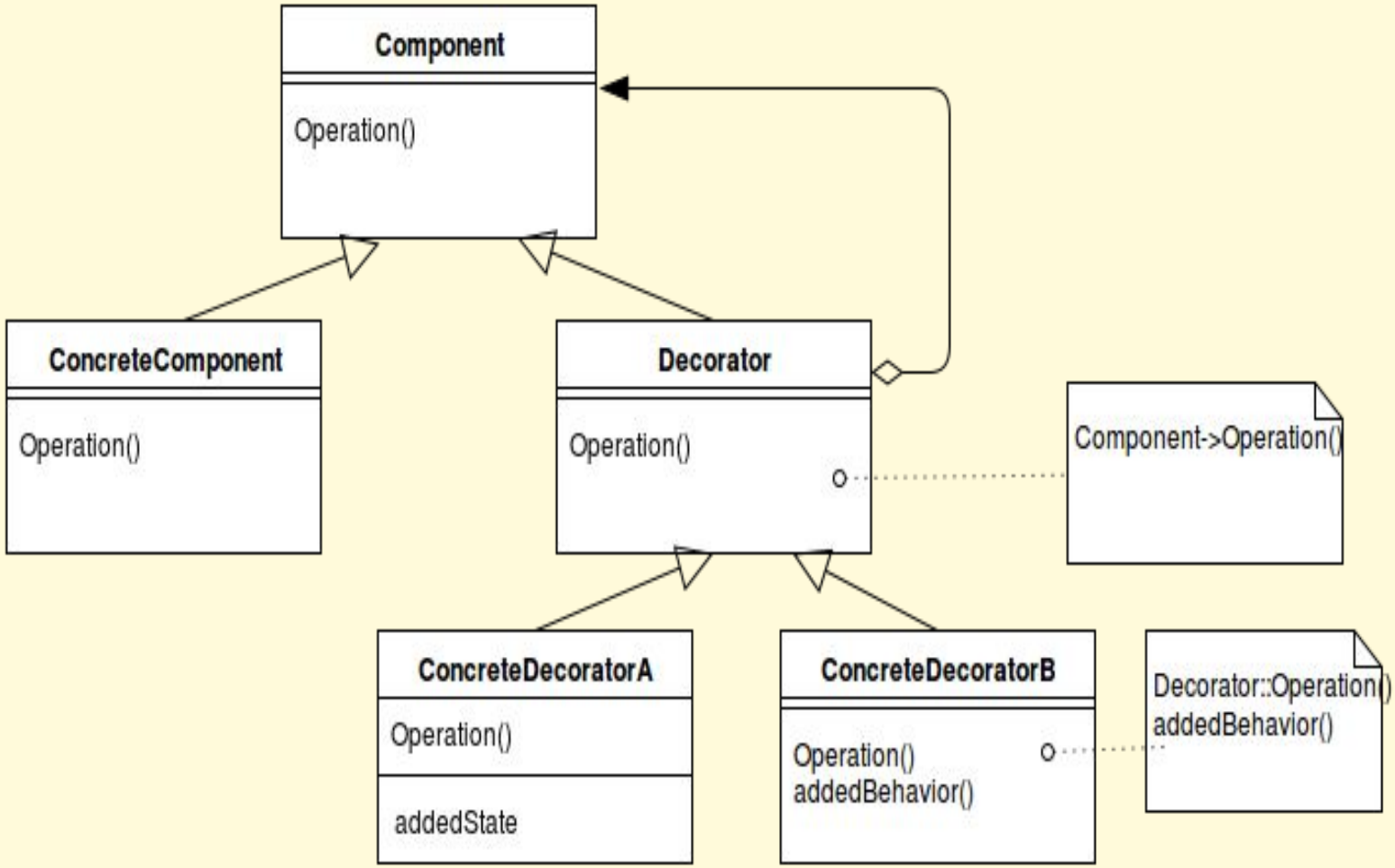
Decorator Pattern

- Structural pattern
- Applies to objects
- Provides flexible alternative to subclass
- Attaches additional responsibilities to objects

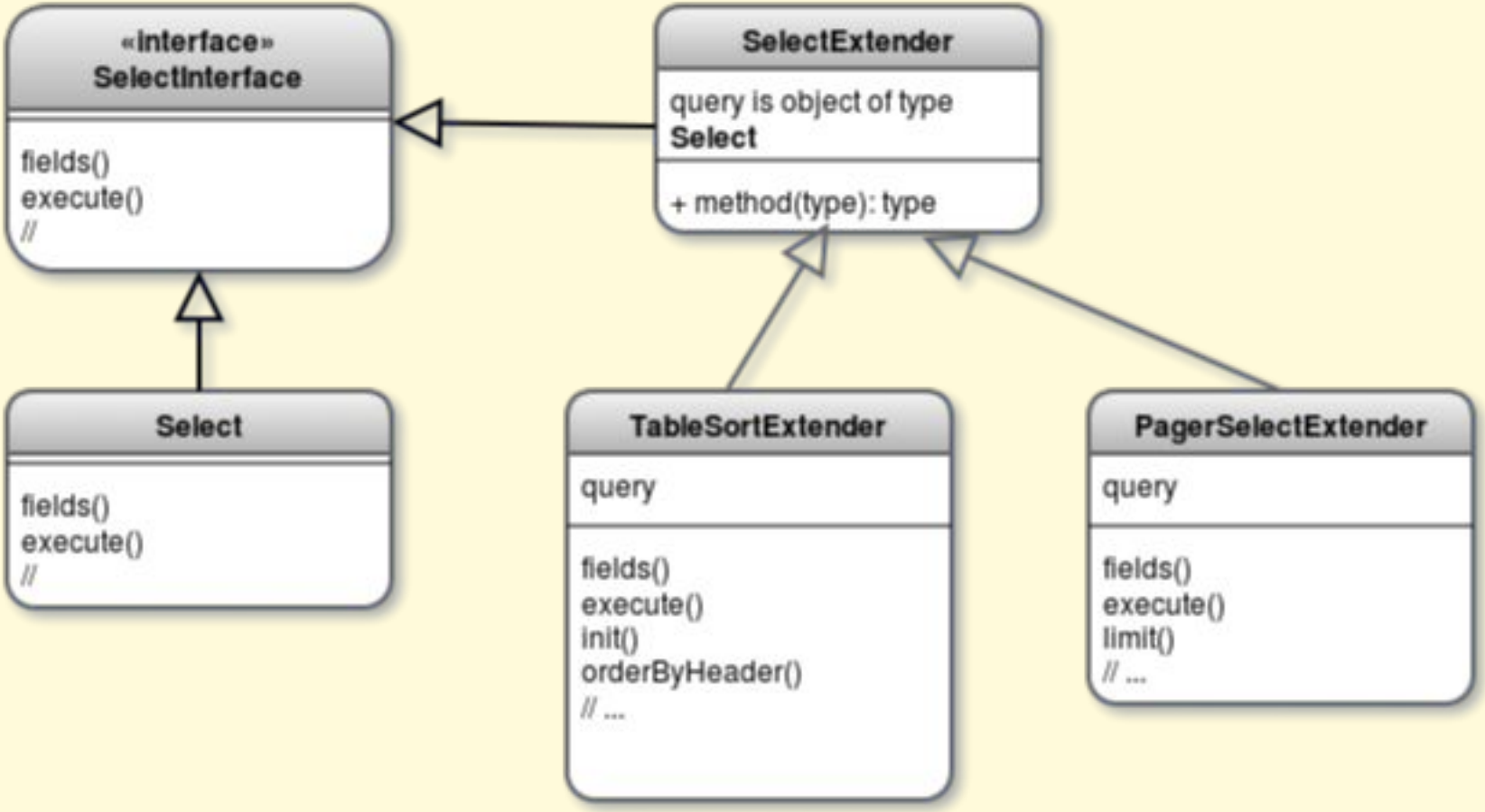
Decorator Pattern



Decorator Pattern



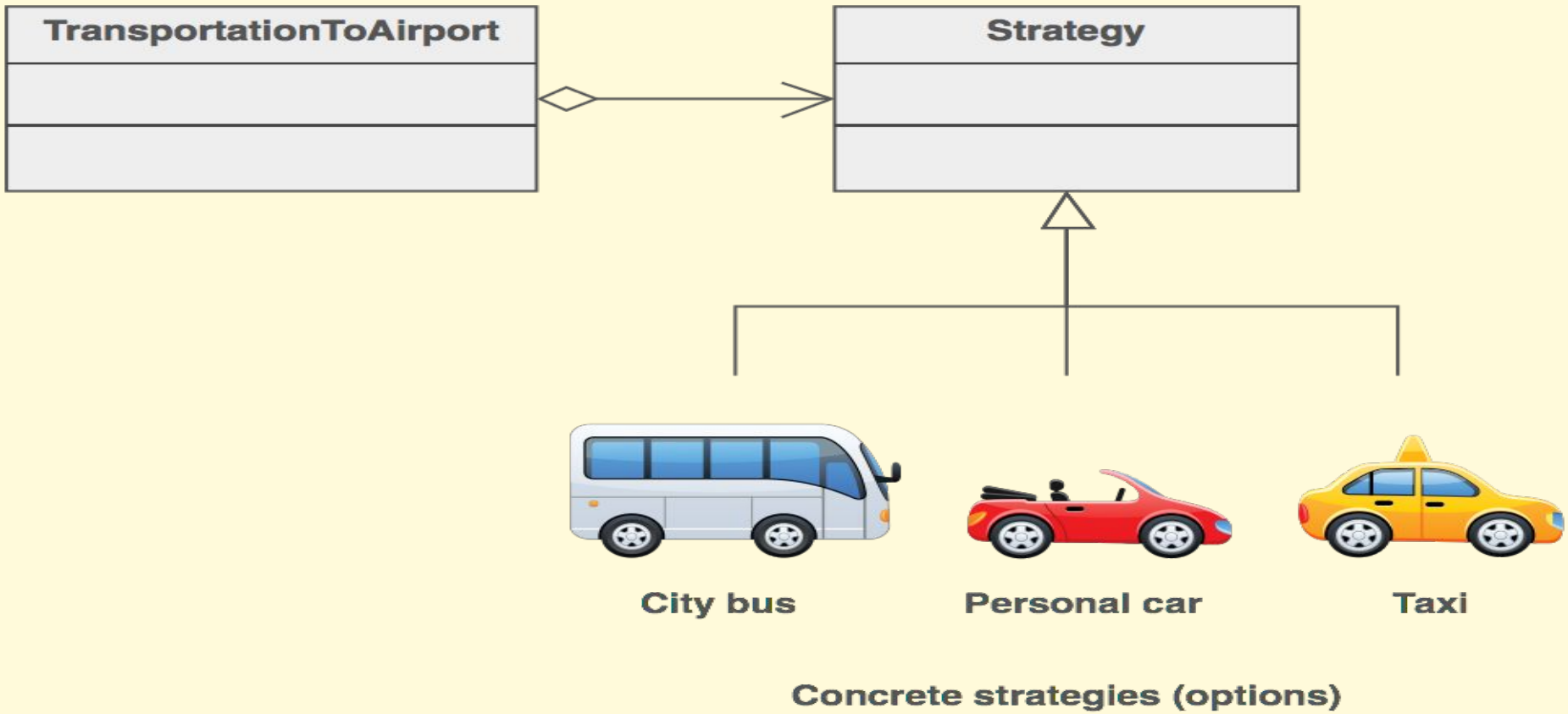
Decorator Pattern D8



Strategy Pattern

- Suggests keeping the implementation of each of algorithms in a separate class
- Each such algorithm encapsulated in a separate class is referred to as a strategy
- Strategy objects must be designed to offer the same interface

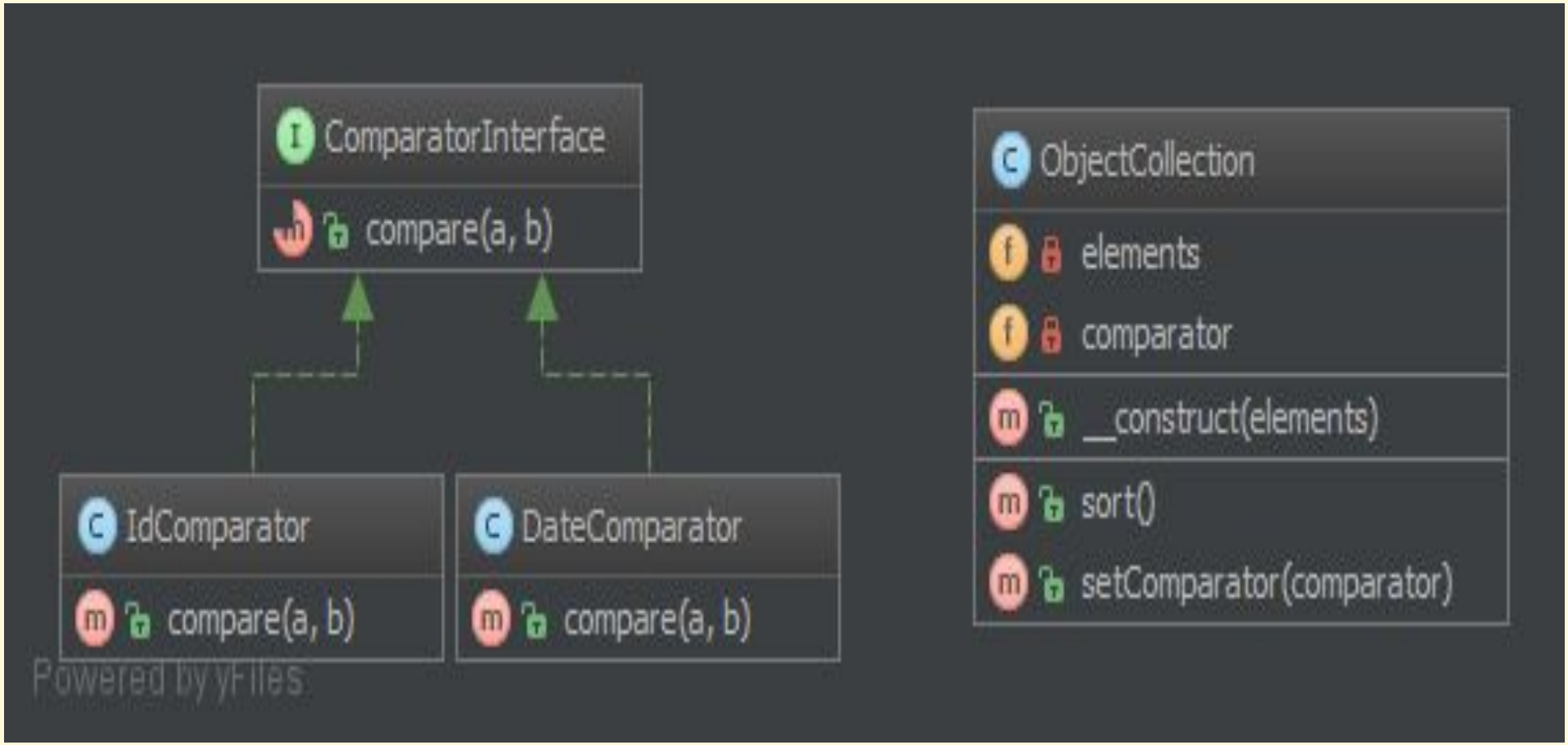
Strategy Pattern



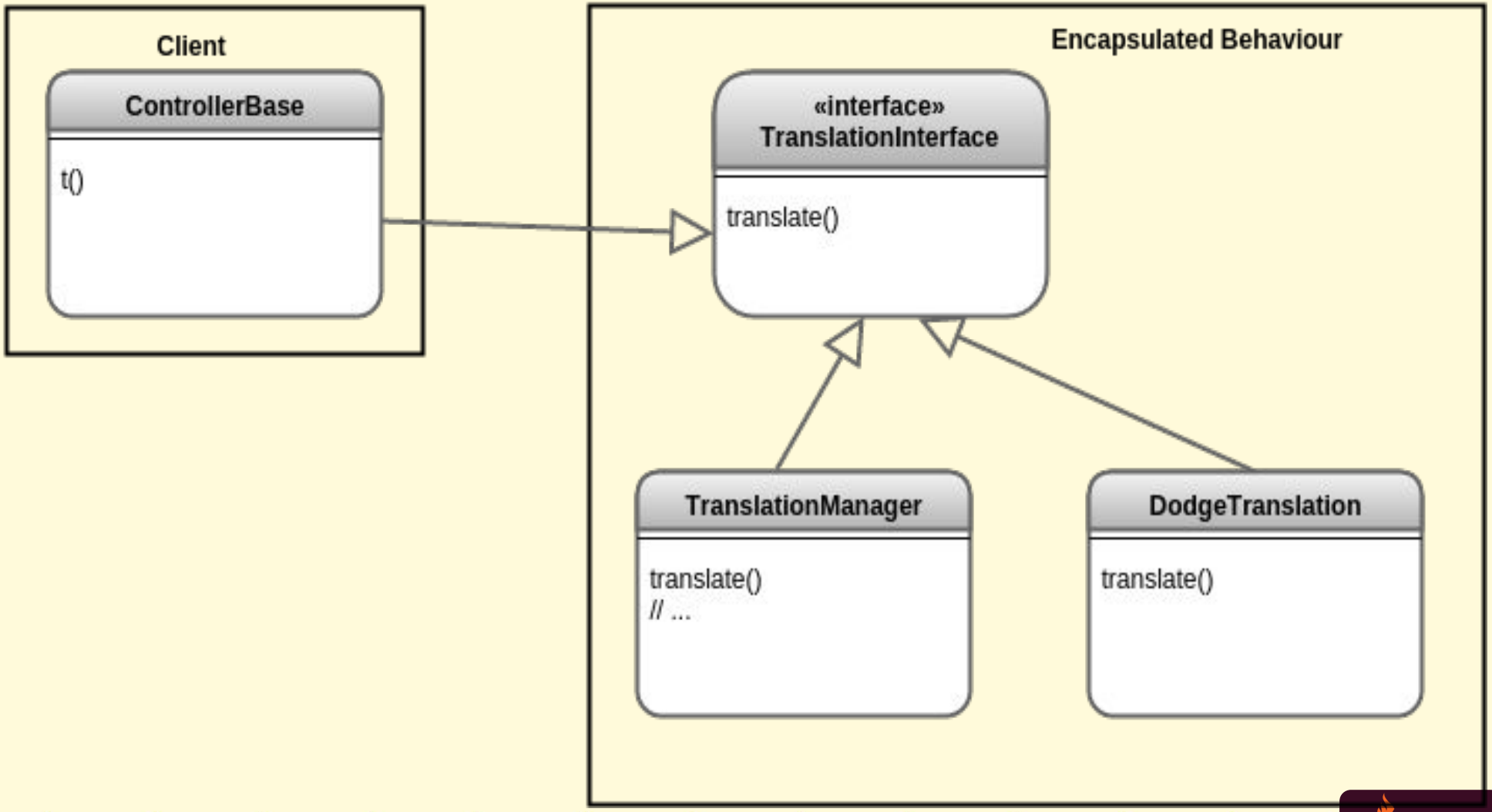
Strategy Pattern

- Behaviors of a class should not be inherited.
- Splits the behavior of a class from the class itself
- Example (Car class)

Strategy Pattern



Strategy Pattern in D8



axelerant

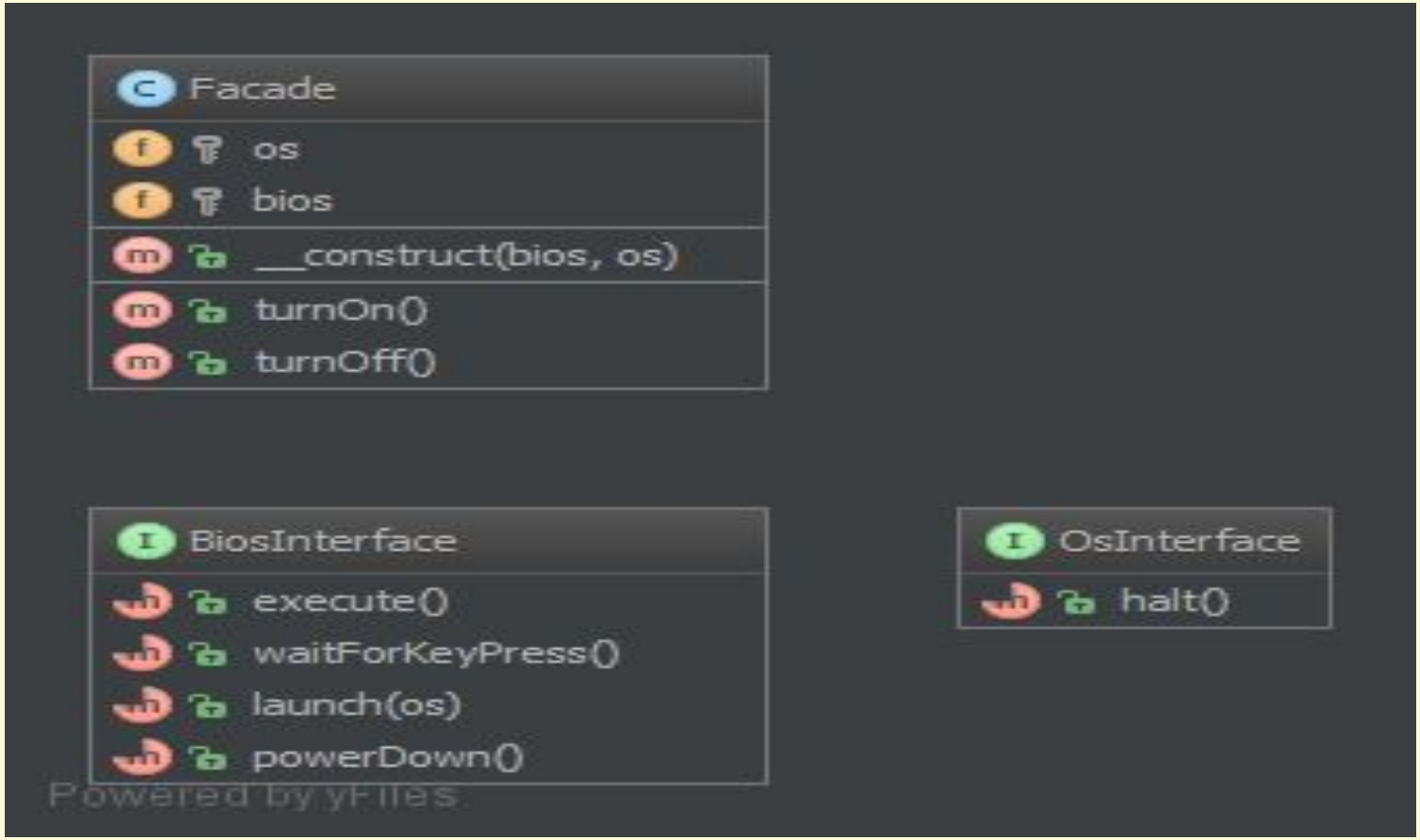
Facade Pattern

- Structural pattern
- Shields the user from the complex details of the system
- Object that provides a simplified interface to a larger body of code
- Example (library)

Facade Pattern

- Defines a unified, higher level interface to a subsystem
- Consumers encounter a Facade when ordering from a catalog
- Has no new and a constructor with interface-type-hinted parameters

Facade Pattern



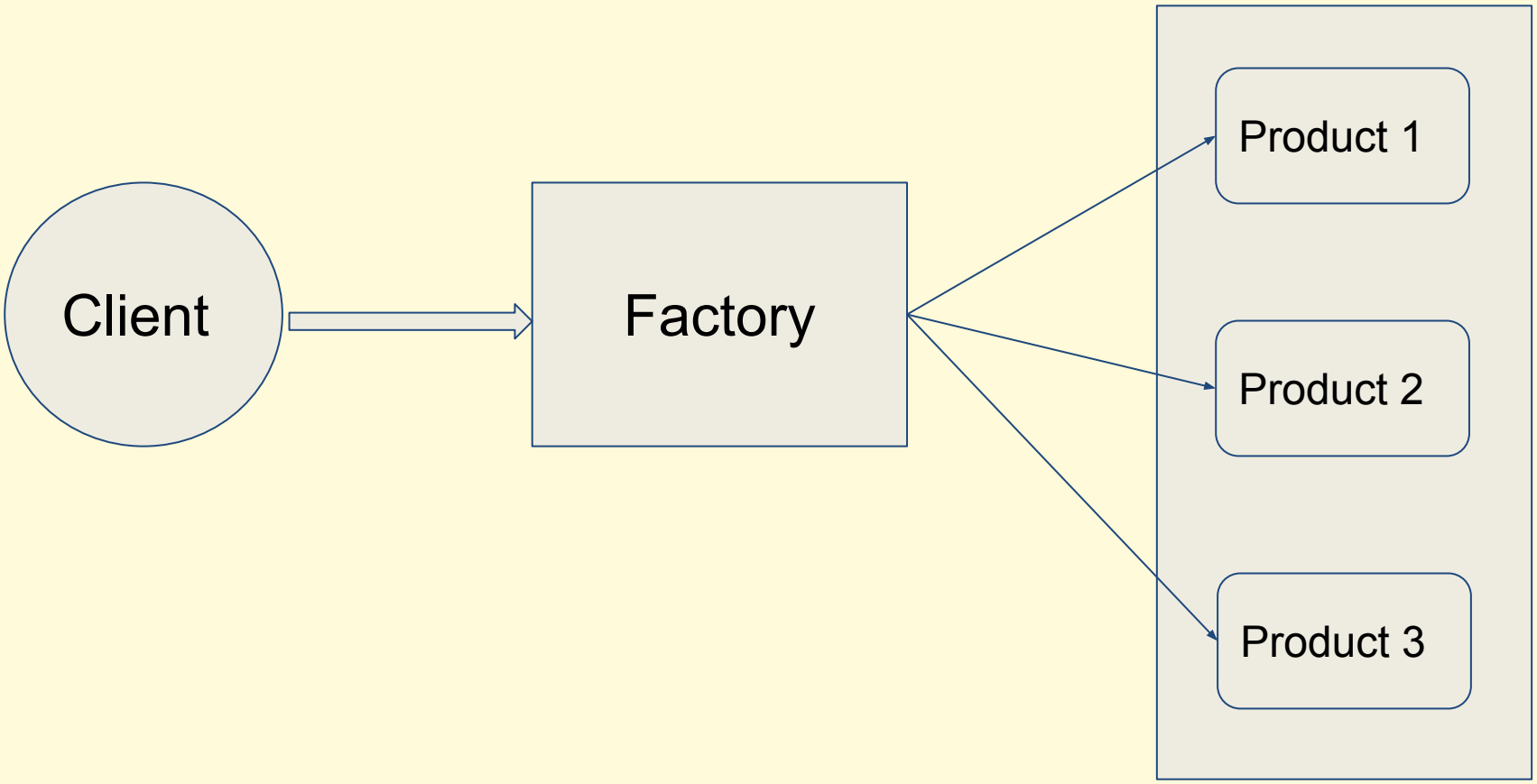
Factory Method Pattern

- Creational pattern
- Define a runtime interface for creating an object
- Also known as "virtual" constructor
- Example (Hotel)

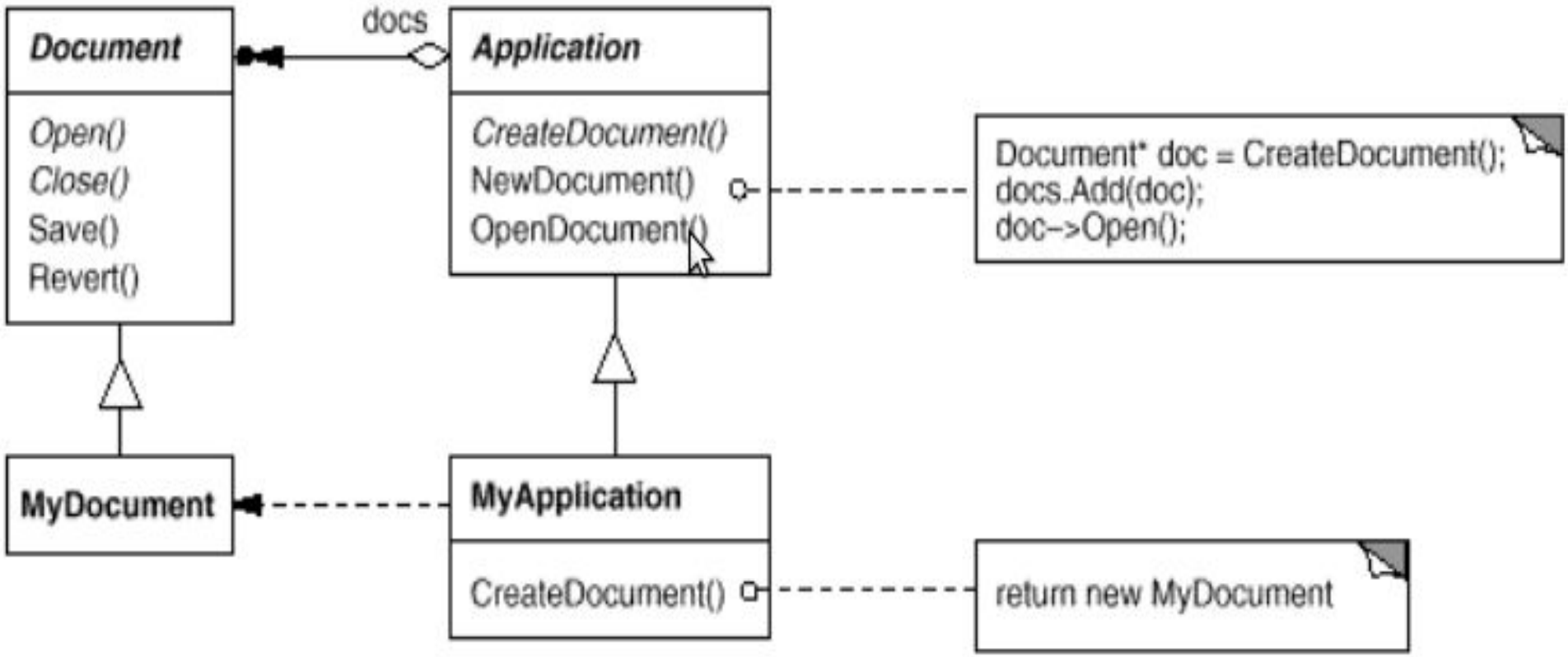
Factory Method Pattern

- Create an abstract class or an interface that declares the factory method
- Create a concrete creator class with default implementation for the factory method in it
- Benefits and drawbacks

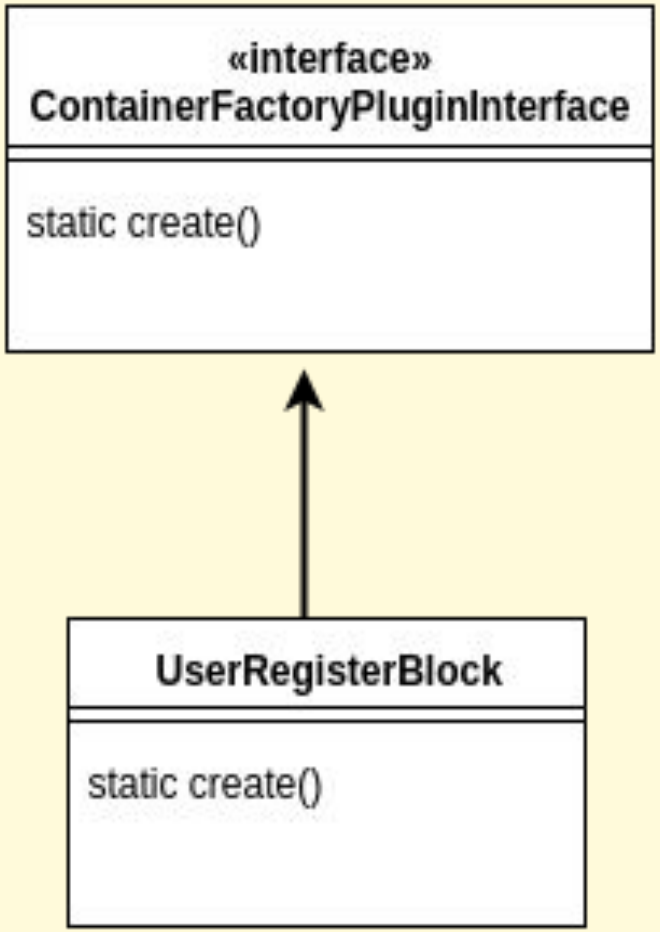
Factory Method Pattern



Factory Method Pattern



Factory Method Pattern in D8



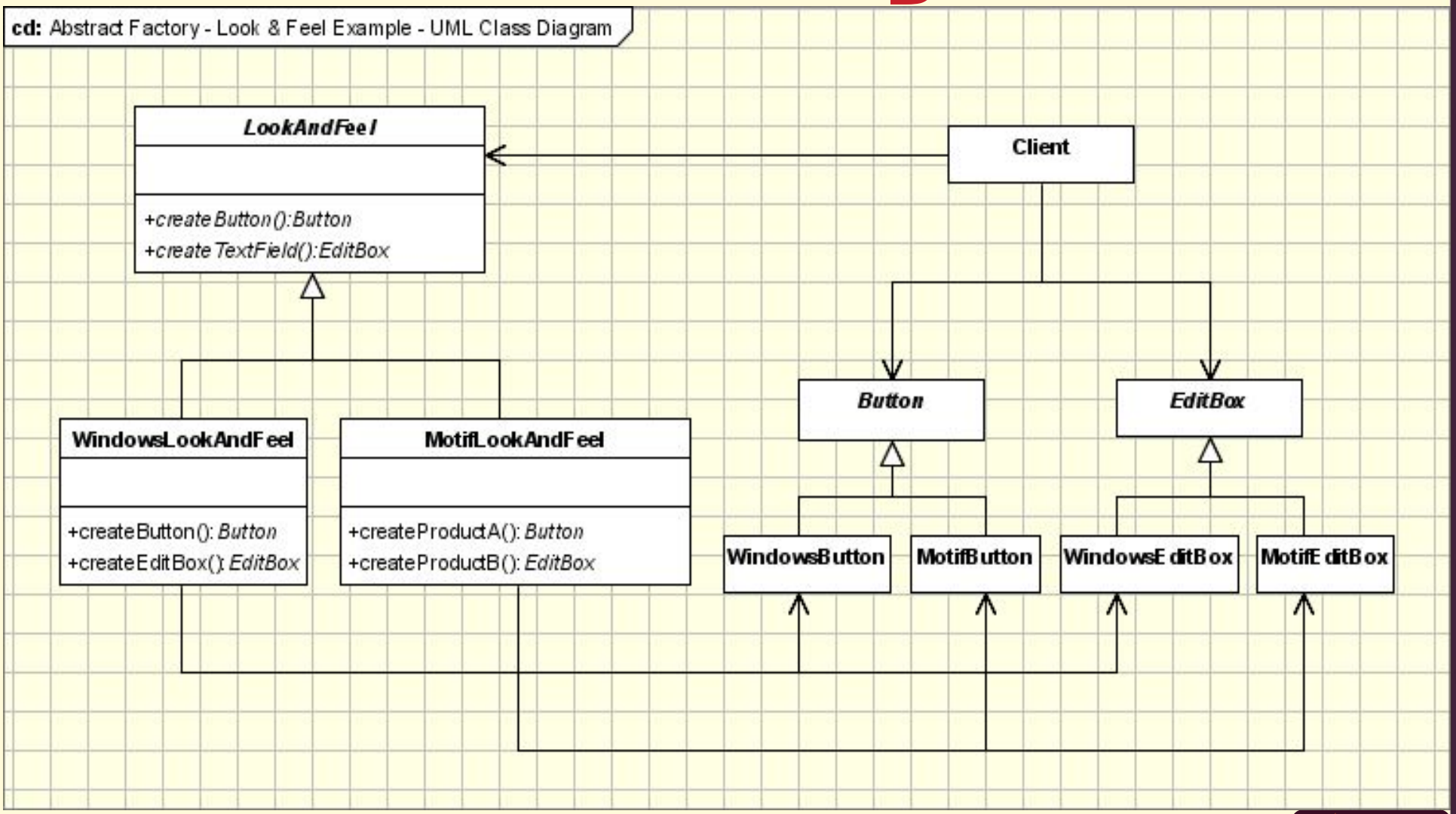
Abstract Factory Pattern

- Creational pattern
- Determines the actual type of the concrete object and creates it
- Returns an abstract pointer to the created object means
- Example (SQL Database and a XML file)

Abstract Factory Pattern

- Created classes all implement the same interface
- Declares only an interface for creating the products

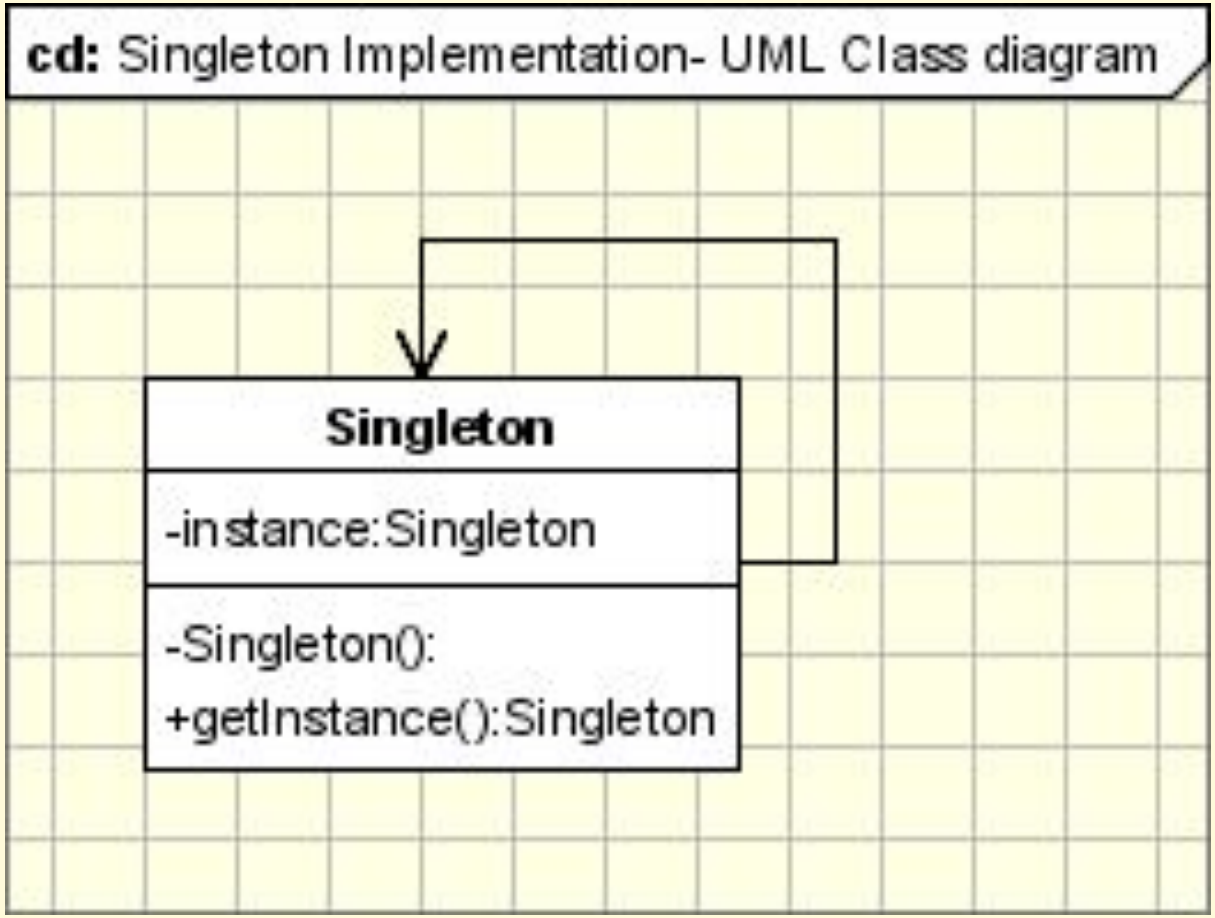
Abstract Factory Pattern



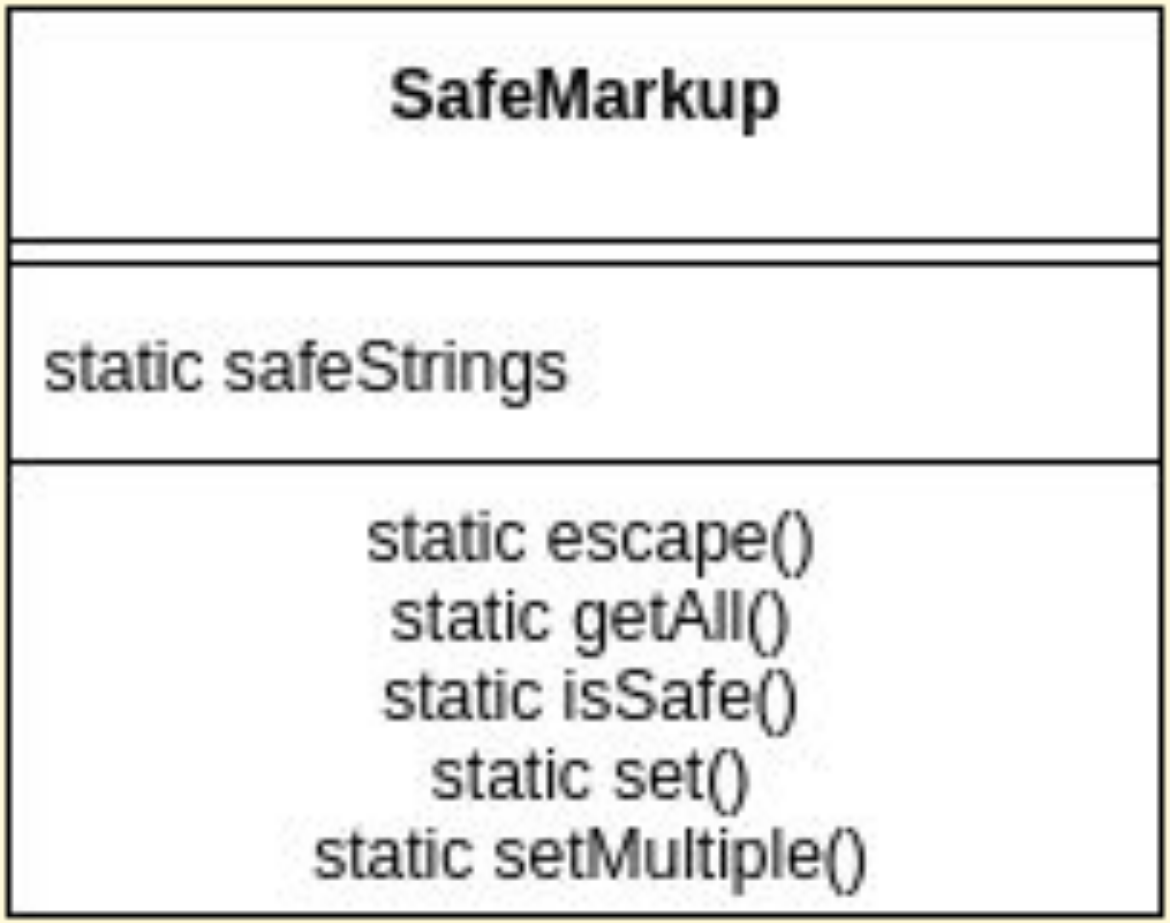
Singleton Pattern

- Creational pattern
- Used to have only one instance of object in the application
- Provide a global point of access to the object.

Singleton Pattern



Singleton Pattern in D8



Singleton Pattern

- Controlled access to sole instance
- Reduced name space
- Permits a variable number of instances
- Example (Logger Classes, Configuration classes)

"Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing the same thing twice."

- Christopher Alexander

Thank you !!

WHAT DID YOU THINK?

EVALUATE THIS SESSION

asia2016.drupal.org/schedule